Seminar 3

Last time: tactics that build the term you would already have written by hand. Today: a tactic that finds a term you would **not** easily write by hand — `induction`.

`MyNat` first, so the tactic's term is small enough to read in full. Then `Nat` — the type you'll use for the rest of the course.

`add_zero` is `rfl`. Is `zero_add`?

```
def add : MyNat → MyNat → MyNat
  | n, .zero   => n
  | n, .succ m => .succ (add n m)

theorem add_zero (n : MyNat) : add n .zero = n := rfl

theorem zero_add (n : MyNat) : add .zero n = n := by
  rfl
```

the failure

```
error: rfl failed
  zero.add n
is not defeq to
  n
```

`add` recurses on its second argument. With `n` a variable, `zero.add n` has nothing to pattern-match on — it's stuck, not equal.

`induction` opens the hypothesis

```
theorem zero_add (n : MyNat) : add .zero n = n := by
  induction n with
  | zero => rfl
  | succ k ih => ?_
```

the `succ` goal

```
k : MyNat
ih : zero.add k = k
⊢ zero.add k.succ = k.succ
```

Two goals, like `cases` — but the second one comes with `ih`, a hypothesis about the smaller case. Induction is `cases` with a hypothesis.

Underneath: `MyNat.rec`, not a proof by magic

```
theorem zero_add (n : MyNat) : add .zero n = n := by
  induction n with
  | zero => rfl
  | succ k ih => rw [add_succ, ih]

#print zero_add
```

```
theorem MyNat.zero_add : ∀ (n : MyNat), zero.add n = n :=
fun n =>
  rec (Eq.refl (zero.add zero))
    (fun k ih =>
      Eq.mpr (id (congrArg (fun _a => _a = k.succ) (add_succ zero k)))
        (Eq.mpr (id (congrArg (fun _a => _a.succ = k.succ) ih)) (Eq.refl k.succ)))
    n
```

`rec` here is `MyNat.rec` — the constant every `induction` on `MyNat` compiles to. The `Eq.mpr` wrapping is what `rw` added; a hand-written term skips it.

New this week

form	what it does
`induction n with | zero => ... | succ k ih => ...`	one goal per constructor, `ih` in the second
`rfl`	closes `a = b` when both sides compute to the same value
`decide`	closes a decidable proposition by computing a `Bool` — no axioms, like `rfl` for facts that aren't syntactic equalities
`Nat.add_succ`, `Nat.succ_add`	the definition of `+`, as citable lemmas
`calc a = b := ... _ = c := ...`	a rewrite chain with every intermediate term spelled out
`simp only [h₁, h₂, ...]`	`rw` with a fixed lemma set, applied until nothing more fires

Round 1 — translate (protected)

`code/Course/Seminar03.lean`, round 1, inside `namespace MyNat`. `double` is defined the same way `add` was — one clause per constructor, `.succ` twice in the second. One fact about it is given as tactics and the term is the exercise; the other is given as a `MyNat.rec` term and the tactics are the exercise.

```
double_add           : double (add n m) = add (double n) (double m)  -- tactics given
double_add_term      : the same fact, as a MyNat.rec term            -- write it

double_eq_add_self_term : double n = add n n, as a MyNat.rec term    -- term given
double_eq_add_self      : the same fact, with tactics                -- write it
```

`double_add`, given

```
theorem double_add (n m : MyNat) :
    double (add n m) = add (double n) (double m) := by
  induction m with
  | zero => rfl
  | succ k ih =>
    rw [add_succ, double_succ, ih,
        double_succ, add_succ, add_succ]
```

the `succ` case, midway

```
n k : MyNat
ih : (n.add k).double = n.double.add k.double
⊢ (n.add k.succ).double = n.double.add k.succ.double
-- after add_succ, double_succ, ih:
⊢ (n.double.add k.double).succ.succ
    = n.double.add k.succ.double
```

Both sides unfold two `.succ`s deep before `ih` applies; the last two `add_succ`s re-fold the right side to match.

`#print double_add`

```
theorem Seminar03.MyNat.double_add :
    ∀ (n m : MyNat), (n.add m).double = n.double.add m.double :=
fun n m =>
  rec (Eq.refl (n.add zero).double)
    (fun k ih =>
      Eq.mpr (id (congrArg (fun _a => _a.double = n.double.add k.succ.double) (add_succ n k)))
        (Eq.mpr (id (congrArg (fun _a => _a = n.double.add k.succ.double) (double_succ (n.add k))))
          (Eq.mpr (id (congrArg (fun _a => _a.succ.succ = n.double.add k.succ.double) ih))
            (Eq.mpr (id (congrArg (fun _a => (n.double.add k.double).succ.succ = n.double.add _a) (double_succ k)))
              (Eq.mpr (id (congrArg (fun _a => (n.double.add k.double).succ.succ = _a) (add_succ n.double k.double.succ)))
                (Eq.mpr (id (congrArg (fun _a => (n.double.add k.double).succ.succ = _a.succ) (add_succ n.double k.double)))
                  (Eq.refl (n.double.add k.double).succ.succ)))))))
    m
```

Six `rw`s, six layers of `Eq.mpr`. Write `double_add_term` yourself: strip it down to the `rec` call underneath.

`double_add_term`, the answer

```
theorem double_add_term (n m : MyNat) :
    double (add n m) = add (double n) (double m) :=
  MyNat.rec
    (motive := fun m => double (add n m) = add (double n) (double m))
    rfl
    (fun _ ih => congrArg MyNat.succ (congrArg MyNat.succ ih))
    m
```

Base case: both sides unfold to `double n`, so `rfl`. Successor case: `double` wraps `.succ` twice, so `ih` needs `congrArg MyNat.succ` applied twice, not once.

`#print double_add_term`

```
theorem Seminar03.MyNat.double_add_term :
    ∀ (n m : MyNat), (n.add m).double = n.double.add m.double :=
fun n m => rec rfl (fun x ih => congrArg succ (congrArg succ ih)) m
```

> **Compare with slide 8: same fact, one line instead of seven. Tactics found a working proof; they didn't find the *shortest* one.**

`double_eq_add_self_term`, given

```
theorem double_eq_add_self_term (n : MyNat) :
    double n = add n n :=
  MyNat.rec (motive := fun n => double n = add n n) rfl
    (fun k ih =>
      (congrArg MyNat.succ (congrArg MyNat.succ ih)).trans
        ((add_succ k.succ k).trans
          (congrArg MyNat.succ (succ_add k k))).symm)
    n
```

Not every step here is unfolding a definition — `succ_add` is a proved lemma, so the term has to route through it with `.trans` / `.symm` instead of a bare `congrArg`. This is what a term looks like once a step needs real work.

`double_eq_add_self`, built up

```
theorem double_eq_add_self (n : MyNat) :
    double n = add n n := by
  induction n with
  | zero => rfl
  | succ k ih =>
    rw [double_succ, ih, add_succ, succ_add]
```

the `succ` case, one rewrite at a time

```
ih : k.double = k.add k
⊢ k.succ.double = k.succ.add k.succ
-- rw [double_succ]
⊢ k.double.succ.succ = k.succ.add k.succ
-- rw [ih]
⊢ (k.add k).succ.succ = k.succ.add k.succ
-- rw [add_succ]
⊢ (k.add k).succ.succ = (k.succ.add k).succ
-- rw [succ_add] rewrites k.succ.add k to
-- (k.add k).succ on the right — both sides match
```

As tactics, citing `succ_add` is just another `rw` — the term on slide 11 is where the cost of that citation actually shows up.

Round 2 — `rfl` or induction

Predict first, then prove. Same shape as slides 2–3: some of these are stuck on a variable, some aren't.

```
myAdd n 0 = n          pow2 3 = 8
myAdd 0 n = n          pow2 (n + 1) = pow2 n + pow2 n
```

`myAdd` and `pow2` are defined the same way `add` was — recursion on the second argument, one clause per constructor.

Round 2 — the answers

goal	`rfl`?	closes with
`myAdd n 0 = n`	yes	`rfl` — matches `myAdd`'s own first clause
`myAdd 0 n = n`	no	`induction n`, `rw [myAdd_succ, ih]`
`pow2 3 = 8`	yes	`rfl` (or `decide`) — three unfoldings and a computation
`pow2 (n + 1) = pow2 n + pow2 n`	yes	`rfl` — it's `pow2`'s own second clause

Every stuck case above is stuck for the same reason as slide 2: the function recurses on the argument that's a bare variable.

Round 3 — own functions on `Nat`

```
mulRec     : own multiplication, by recursion          -- define it
mulRec_eq  : mulRec n m = n * m                         -- prove it

double_double : double (double n) = 4 * n               -- no induction
pow2_add      : pow2 (n + m) = pow2 n * pow2 m          -- induction
```

`double` and `pow2` are the functions from rounds 1–2; `double_eq_two_mul : double n = 2 * n` was proved in lecture and is given again at the top of this section of the file.

`mulRec`, defined and proved

```
def mulRec : Nat → Nat → Nat
  | _, 0     => 0
  | n, m + 1 => mulRec n m + n

theorem mulRec_eq (n m : Nat) : mulRec n m = n * m := by
  induction m with
  | zero => rfl
  | succ k ih => rw [mulRec_succ, ih, Nat.mul_succ]
```

the `succ` case

```
n k : Nat
ih : mulRec n k = n * k
⊢ mulRec n (k + 1) = n * (k + 1)
```

Same recursion as `add`, same proof shape as `double_eq_two_mul`: unfold, cite `ih`, close with the one `Nat.mul_succ` that matches.

`double_double`, two rewrites

```
theorem double_double (n : Nat) :
    double (double n) = 4 * n := by
  rw [double_eq_two_mul, double_eq_two_mul, ← Nat.mul_assoc]
```

the goal, rewritten three times

```
⊢ double (double n) = 4 * n
⊢ 2 * double n = 4 * n         -- rw [double_eq_two_mul]
⊢ 2 * (2 * n) = 4 * n          -- rw [double_eq_two_mul] again
-- rw [← Nat.mul_assoc] : 2 * 2 * n = 4 * n, closes by computation
```

No `induction` — `double_eq_two_mul` already did the recursion, twice over. This is what "prove a function correct, then reuse the proof" buys you.

`pow2_add`, induction on `m`

```
theorem pow2_add (n m : Nat) :
    pow2 (n + m) = pow2 n * pow2 m := by
  induction m with
  | zero =>
      have h0 : pow2 0 = 1 := rfl
      rw [Nat.add_zero, h0, Nat.mul_one]
  | succ k ih =>
      rw [Nat.add_succ, pow2_succ, ih,
          ← Nat.mul_add, ← pow2_succ]
```

both cases

```
-- zero: pow2 (n + 0) = pow2 n * pow2 0
-- succ, before rw, with
--   ih : pow2 (n + k) = pow2 n * pow2 k
⊢ pow2 (n + (k + 1)) = pow2 n * pow2 (k + 1)
```

`pow2_succ` forwards splits the left side into two copies of `pow2 (n + k)`; `ih` turns those into two copies of `pow2 n * pow2 k`; `Nat.mul_add` and `pow2_succ` backwards fold them into `pow2 n * pow2 (k + 1)`.

What to take away

- **Induction is `cases` with a hypothesis.** The extra goal comes with `ih`.
- `rfl` closes what's stuck on a literal; a variable in the wrong spot gets you nowhere until you induct on it.
- `#print` after `induction` shows `T.rec` wearing `Eq.mpr` from every `rw` — a hand-written term is the same recursor, undressed.
- `Nat.add_succ`, `Nat.mul_succ`, ... are a definition's own equations, citable like any other lemma.
- Proving a function correct once (`double_eq_two_mul`) pays for itself: reusing it beats re-inducting.
- A term is cheap when every step is unfolding a definition, and expensive the moment it needs an actual lemma — `double_eq_add_self_term` versus its four-line tactic proof was the whole difference.